DEEP LEARNING PRACTICAL 02

ADULT INCOME PREDICTION USING ARTIFICIAL NEURAL NETWORKS

Project	Adult Income Prediction Using ANN
Dataset	UCI Adult / Census Income Dataset
Learning Task	Binary Classification
Model Type	Artificial Neural Network (Multilayer Perceptron)

Project Report

Abstract

This practical develops and evaluates an Artificial Neural Network (ANN) for predicting whether an individual's annual income is above or at/below $50,000 using demographic and employment-related attributes from the Adult Income dataset. The work covers data inspection, cleaning, exploratory analysis, preprocessing, baseline model development, and systematic comparisons of activation functions, weight initializers, loss functions, batch normalization, and optimizers. The final optimized model is evaluated using classification metrics, including accuracy, precision, recall, F1-score, and ROC-AUC.

1. Introduction

Income classification is a binary classification problem in which the model learns patterns between personal, demographic, and employment characteristics and an income category. An ANN is used because its interconnected layers can learn non-linear relationships among input features. The practical emphasizes both model construction and controlled experimentation with training configurations.

2. Objectives

• Load, inspect, clean, and preprocess the Adult Income dataset.

• Explore the distribution of the target classes and relationships among selected features.

• Build a baseline multilayer perceptron for binary classification.

• Compare activation functions, weight initializers, loss functions, batch normalization, and optimizers.

• Select and evaluate a final ANN configuration using multiple performance metrics.

3. Dataset Description

The notebook records 48,842 original observations. Rows containing missing values represented by '?' were removed, leaving 45,222 cleaned records. The columns 'fnlwgt' and 'education' were removed as specified in the practical. The target variable 'income' was encoded as 0 for <=50K and 1 for >50K. The processed feature matrix contained 80 input features after categorical encoding.

4. Data Preprocessing and Exploratory Data Analysis

Categorical values were stripped of surrounding whitespace and '?' markers were treated as missing values. Rows with missing values were dropped. The target classes were converted to binary labels. Categorical predictors were one-hot encoded, while numerical predictors were standardized using StandardScaler. The notebook uses a stratified 80:20 train-test split with random_state=42. The recorded final split contains 36,177 training samples and 9,045 test samples; a validation subset is used during model training.

The EDA work includes income class distribution, age distribution by income, weekly working-hours boxplots, education-level counts, and a numerical correlation heatmap. These visualizations are included separately in the project plots archive.

5. Model Development and Experiments

A reusable ANN builder was created to vary hidden-layer settings, activation, initializer, batch normalization, optimizer, and loss. The baseline architecture uses two hidden layers with 128 and 64 neurons and a single sigmoid output neuron. The experiment stages compare the specified alternatives and use validation performance to guide configuration selection.

Task 2 – Baseline ANN

A baseline multilayer perceptron was trained as the reference model for subsequent experiments.

Task 3 – Activation Functions

Different hidden-layer activation functions were compared to study their effect on learning and validation performance.

Task 4 – Weight Initializers

Alternative kernel initializers were evaluated to understand their influence on optimization and convergence.

Task 5 – Loss Functions

Loss functions for binary classification were compared, including the use of a focal-style loss to address class imbalance.

Task 6 – Batch Normalization

Batch normalization configurations were tested to assess their effect on training stability and validation results.

Task 7 – Optimizers

Optimizer and learning-rate configurations were compared. The recorded selected configuration was Adam with a learning rate of 0.0001.

6. Final Optimized ANN Configuration

Component	Recorded configuration
Input layer	80 input features
Hidden layer 1	128 neurons; ELU activation
Hidden layer 2	64 neurons; ELU activation
Output layer	1 neuron; sigmoid activation
Weight initialization	He Uniform
Loss function	Focal Loss
Optimizer	Adam
Learning rate	0.0001
Batch normalization	Disabled in final architecture
Batch size	64
Maximum epochs	80 (early stopping used)
Parameters	18,689

7. Final Model Performance

The notebook's recorded final test results are summarized below. These values are reported from the saved notebook outputs.

Metric	Test result
Accuracy	0.8169 (81.69%)
Precision	0.5938
Recall	0.8274
F1-score	0.6914
ROC-AUC	0.9084

The ROC-AUC of 0.9084 indicates strong ranking ability in distinguishing the two income classes. Recall (0.8274) is higher than precision (0.5938), showing that the model identifies a substantial proportion of higher-income cases, while some positive predictions are false positives. The F1-score balances precision and recall.

8. Conclusion

This practical demonstrates a complete ANN-based workflow for Adult Income classification, from data preparation and EDA to model experimentation and evaluation. The final recorded configuration achieved 81.69% accuracy and 0.9084 ROC-AUC on the test set. The experiments show why model configuration should be assessed using several metrics rather than accuracy alone, particularly when the target classes are imbalanced.

9. Project Files

• Deep_Learning_Project_2.ipynb – notebook containing code, experiments, outputs, and visualizations.

• Deep_Learning_Practical_02_Adult_Income_Prediction_Using_ANN.html – static HTML export of the notebook.

• adult.csv – dataset file used for the practical.

• plots.zip – exported visualizations.

• results.zip – exported result tables.

• requirements.txt – Python package dependencies.

Note: The report describes the recorded notebook results. A trained .keras model file is a separate artifact and is not included in this report unless it is separately generated and added to the repository.